

Day 4: YOLO — Interview Revision

Object Detection, YOLO Architecture, IoU, NMS, Loss Functions, Multi-Scale Detection and Segmentation

1. Core YOLO Idea

YOLO (You Only Look Once) is a one-stage object detector. It processes an image through a neural network and directly predicts candidate bounding boxes, objectness/confidence scores, and class scores. Post-processing such as confidence filtering and Non-Maximum Suppression (NMS) is then used to obtain the final detections.

Image → Backbone → Feature Maps → Detection Heads → Candidate Predictions → Confidence Filtering → NMS → Final Detections

Question 1

What is the difference between image classification and object detection?

Answer: Image classification assigns an entire image to one or more classes. Object detection identifies what objects are present and also localizes each object using bounding boxes.

Related Tasks

Task	Output
Classification	What is in the image? → Class label
Object Detection	What + Where? → Class + Bounding Box
Semantic Segmentation	Class label for every pixel
Instance Segmentation	Pixel-level mask for each individual object

Question 2

Why is YOLO called a "one-stage" detector?

Answer: YOLO directly predicts bounding boxes, objectness scores, and class scores from the image in a single forward pass. Unlike two-stage detectors, it does not first generate region proposals and then separately classify and refine those regions.

YOLO: Image → Direct Predictions → Post-processing → Final Detections

Two-stage detector: Image → Region Proposals → Classification/Refinement → Detections

Question 3

How can YOLO detect multiple objects if it processes the image in one forward pass?

Answer: One forward pass does not mean one prediction. The network generates many candidate predictions across the image. These predictions contain bounding boxes, objectness/confidence scores, and class scores. After confidence filtering and NMS, multiple final objects can remain.

One Image → One Forward Pass → Many Candidate Predictions → Filtering + NMS → Multiple Final Detections

Question 4

What does a YOLO prediction contain?

Answer: A YOLO prediction generally contains bounding-box parameters, an objectness/confidence score, and class scores. The bounding-box parameters describe the location and size of the predicted box. Objectness indicates whether the prediction contains an object, while class scores indicate which class the object belongs to.

Prediction $\approx$ (x, y, w, h, objectness, class scores)

Note: The exact parameterization and output format varies across YOLO versions. Do not assume every YOLO version uses exactly the same prediction representation.

Question 5

What is IoU and why is it important?

Answer: IoU (Intersection over Union) measures the overlap between two bounding boxes. During training or evaluation, it can compare a predicted box with a ground-truth box. During NMS, it is used to measure overlap between predicted boxes.

$\text{IoU} = \text{Area of Intersection} / \text{Area of Union}$

IoU = 1 means perfect overlap. IoU = 0 means no overlap. Higher IoU generally means better localization when comparing a prediction with its ground truth.

Question 6

How does YOLO know which prediction is responsible for a ground-truth object?

Answer: In the original YOLO formulation, the center of the ground-truth object determines the responsible grid cell. That cell predicts multiple candidate boxes, and the prediction with the highest IoU with the ground-truth box is assigned responsibility for that object.

The responsible prediction is trained using the relevant localization, objectness, and classification losses. Other predictions are handled as non-responsible/background predictions.

Modern YOLO note: Modern YOLO versions can use more sophisticated label-assignment strategies, so the exact original YOLOv1 matching rule should not be assumed for every modern version.

Question 7

What are the main things YOLO needs to learn?

Answer: YOLO needs to learn three main things: where the object is, whether an object is present, and what class the object belongs to.

Where? → Bounding-box localization (x, y, w, h)

Is there an object? → Objectness

What is it? → Classification

Question 8

Why does YOLO need a no-objectness loss?

Answer: Most candidate predictions correspond to background rather than real objects. The no-objectness loss penalizes predictions that incorrectly assign high objectness to background boxes. This teaches the model not to predict objects everywhere.

Question 9

Why is the no-objectness loss weighted differently?

Answer: Background predictions vastly outnumber predictions responsible for real objects. If all background predictions contributed equally with a large weight, the loss could be dominated by background. The model might learn to predict no object everywhere. Therefore, the original YOLO loss downweighted the no-objectness term and increased the importance of localization.

$$L = \lambda_{\text{coord}} L_{\text{localization}} + L_{\text{objectness}} + \lambda_{\text{noobj}} L_{\text{no-objectness}} + L_{\text{classification}}$$

In the original YOLO formulation, λ_{coord} was used to emphasize localization, while λ_{noobj} was used to reduce the contribution of background predictions.

Question 10

What is the difference between the confidence threshold and the NMS IoU threshold?

Answer: The confidence threshold filters out weak predictions. The NMS IoU threshold determines how much overlap between predicted boxes is considered excessive when removing duplicate detections.

Threshold	Question it answers	Purpose
Confidence threshold	Is this prediction confident enough?	Remove weak predictions
NMS IoU threshold	Is this box too similar to another box?	Suppress duplicate overlapping boxes

Question 11

How does NMS work?

Answer: First, weak predictions are removed using a confidence threshold. The remaining predictions are sorted by confidence. The highest-confidence box is kept, and boxes with excessive IoU overlap are suppressed. The process is repeated with the remaining boxes until the final detections are obtained.

Predictions → Confidence Filtering → Sort by Confidence → Keep Highest → Suppress High-IoU Duplicates → Repeat

Question 12

What is the difference between IoU and objectness/confidence?

Answer: IoU measures geometric overlap between two boxes. Objectness/confidence indicates how strongly the model believes a prediction contains an object. IoU can be used for comparing predicted and ground-truth boxes or for comparing predicted boxes during NMS.

IoU → How much do the boxes overlap?

Objectness → Is there an object here?

Class score → What class is the object?

Question 13

What is the difference between anchor-based and anchor-free detection?

Answer: Anchor-based detectors start with predefined bounding-box shapes called anchors and predict adjustments to fit the actual objects. Anchor-free detectors do not rely on predefined anchor shapes and instead directly predict box geometry from prediction locations or points.

Anchor-based: Predefined anchor → Predict offsets → Final box

Anchor-free: Prediction location → Directly predict box geometry

Important: YOLOv1 did not use predefined anchor boxes in the modern anchor-box sense. Later YOLO versions introduced anchor-based approaches, while many modern YOLO versions use anchor-free formulations.

Question 14

Why is YOLO generally faster than two-stage detectors such as Faster R-CNN?

Answer: YOLO is a one-stage detector that directly predicts candidate detections from the image. Faster R-CNN uses a Region Proposal Network to generate candidate regions and then performs additional classification and bounding-box refinement. The extra proposal and refinement stages generally require more computation.

Important: Faster R-CNN is an object detector. The 'R' stands for 'Region', not Recurrent.

Question 15

What is multi-scale detection?

Answer: Multi-scale detection means making predictions using feature maps at multiple spatial resolutions. Higher-resolution feature maps preserve more spatial detail and are generally better for small objects. Lower-resolution feature maps provide more compressed and global information and are generally better for larger objects.

High-resolution feature map → Small objects

Medium-resolution feature map → Medium objects

Low-resolution feature map → Large objects

Question 16

Why are small objects harder to detect?

Answer: Small objects occupy fewer pixels, so their visual features can be lost as the CNN downsamples the image and produces lower-resolution feature maps. In original YOLO, the grid formulation could also make small-object detection difficult. Modern YOLO models use multi-scale feature maps to preserve more spatial detail.

Question 17

What are some limitations of YOLO?

Answer: YOLO can struggle with very small objects, crowded or heavily overlapping objects, and precise localization in some situations. Standard object detection also only provides bounding boxes rather than exact pixel-level object masks. In addition, training requires labeled data, and creating accurate bounding-box annotations can be time-consuming and expensive.

Note: Bounding-box detection not providing exact object boundaries is a limitation of the detection representation itself, not uniquely a YOLO limitation.

Question 18

What is the difference between object detection and instance segmentation?

Answer: Object detection identifies and localizes objects using bounding boxes. Instance segmentation additionally produces a pixel-level mask for each individual object. Instance segmentation is useful when precise object boundaries are important, such as isolating tumors, cells, or other structures in medical images.

Object detection → Class + Bounding Box

Instance segmentation → Class + Bounding Box + Individual Pixel Mask

Question 19

What is the difference between semantic segmentation and instance segmentation?

Answer: Semantic segmentation assigns a class label to every pixel but does not distinguish between individual objects of the same class. Instance segmentation assigns a separate pixel-level mask to each individual object instance.

Semantic segmentation: Dog + Dog + Dog → all pixels labeled 'Dog'

Instance segmentation: Dog 1 + Dog 2 + Dog 3 → separate masks

The individual dogs are different **instances** of the same **class**; they are not different categories.

Question 20

Explain the complete YOLO inference pipeline.

Answer: The image is passed through the backbone to extract features. The network produces feature maps, potentially at multiple scales. Detection heads use these features to predict candidate bounding boxes, objectness/confidence scores, and class scores. Predictions from different scales are combined, weak predictions are filtered, and NMS removes redundant overlapping detections. The remaining predictions are the final detected objects.

Image

- Backbone
- Feature Maps / Multi-scale Features
- Detection Heads
- (x, y, w, h) + Objectness + Class Scores
- Combine Candidate Predictions
- Confidence Filtering
- NMS
- Final Bounding Boxes + Class Labels

Quick YOLO Interview Cheat Sheet

Concept	Remember
YOLO	One-stage object detector
Bounding box	Localizes an object using coordinates
Objectness	How strongly the model believes an object is present
Class score	Which class the object belongs to
IoU	Intersection area divided by union area
Confidence threshold	Removes weak predictions
NMS	Removes redundant overlapping detections
NMS IoU threshold	Controls how much overlap triggers suppression
Localization	Learn where the object is
Objectness	Learn whether an object is present
Classification	Learn what class the object belongs to
No-objectness loss	Penalizes high objectness for background predictions
Anchor-based	Uses predefined anchor shapes and predicts adjustments
Anchor-free	Does not use predefined anchor shapes
Multi-scale detection	Uses different feature-map resolutions
Small objects	Benefit from higher-resolution feature maps
Object detection	Bounding boxes
Instance segmentation	Separate pixel-level mask per object
Semantic segmentation	Class label for each pixel, no instance separation

Core YOLO mental model:

YOLO takes an image, extracts features, makes many candidate predictions across the image and potentially across multiple feature-map scales, predicts where objects are, whether objects are present, and what classes they belong to. Weak predictions are filtered, and NMS removes redundant overlapping detections to produce the final results.